

```
In [3]: # =====
# Notebook setup
# =====

%load_ext autoreload
%autoreload 2

# Retrain or use existing models
use_cached_weights = True

# Control figure size
figsize=(14, 3.5)
figsize_narrow=(14, 2.5)

from util import util
from tensorflow import keras
import warnings
warnings.filterwarnings('ignore')

# Generate a problem instance
nitems, rel_req, seed = 20, 0.5, 42
prb = util.generate_problem(nitems=nitems, rel_req=rel_req, seed=seed)

# Generate basic dataset
data_tr = util.generate_costs(nsamples=350, nitems=nitems, seed=seed, noise_
data_ts = util.generate_costs(nsamples=150, nitems=nitems, seed=seed, sampli
```

The autoreload extension is already loaded. To reload it, use:
`%reload_ext autoreload`

Applicability of DFL

Let's Second-Guess Ourselves

However, let's not discount the prediction-focused approach yet

In fact, it's easy to see that:

$$\mathbb{E}[\text{regret}(y, \hat{y})] \xrightarrow{\mathbb{E}[L(y, \hat{y})] \rightarrow 0} 0$$

Intuitively:

- The more accurate we can be, the lower the regret
- Eventually, perfect predictions will result in 0 regret

But then... What if we make our model bigger?

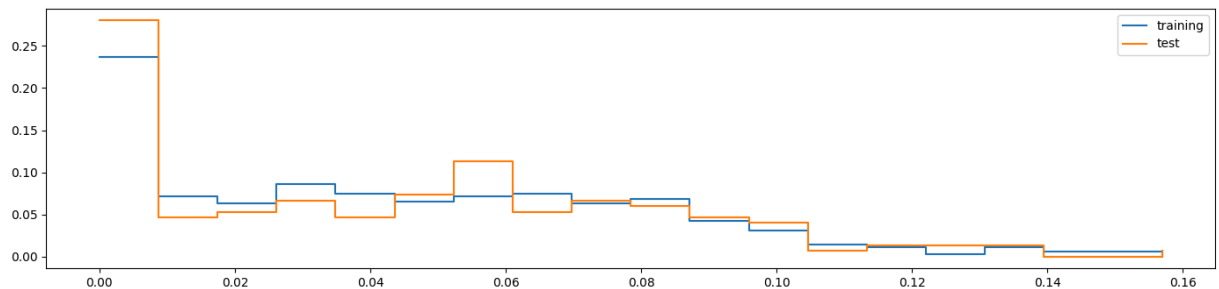
- We could get good predictions *and* good regret

- ...And training would be *much* faster

Our Baseline

Let's check again the results for our PFL linear regressor

```
In [4]: pfl = util.build_ml_model(input_size=1, output_size=nitems, hidden=[], name=
history = util.train_ml_model(pfl, data_tr.index.values, data_tr.values, epo
r_tr = util.compute_regret(prb, pfl, data_tr.index.values, data_tr.values)
r_ts = util.compute_regret(prb, pfl, data_ts.index.values, data_ts.values)
util.plot_histogram(r_tr, figsize=figsize, label='training', data2=r_ts, lat
```



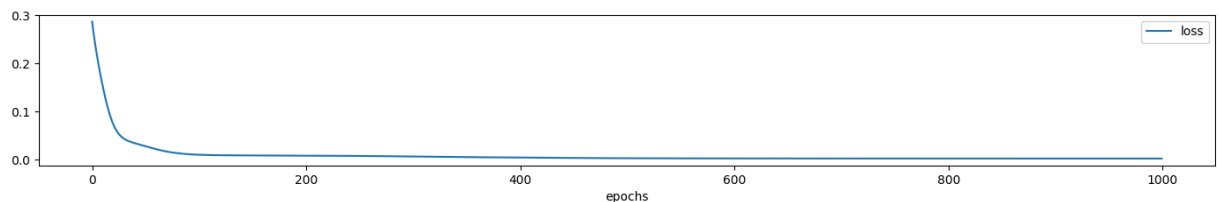
Mean: 0.044 (training), 0.044 (test)

This will be our main baseline

PFL Strikes Back

Let's try to use a *non-linear model*

```
In [5]: pfl_acc = util.build_ml_model(input_size=1, output_size=nitems, hidden=[8],
history = util.train_ml_model(pfl_acc, data_tr.index.values, data_tr.values,
util.plot_training_history(history, figsize=figsize_narrow, print_scores=False)
util.print_ml_metrics(pfl_acc, data_tr.index.values, data_tr.values, label='
util.print_ml_metrics(pfl_acc, data_ts.index.values, data_ts.values, label='
```



Training time: 12.0623 sec

R2: 0.95, MAE: 0.041, RMSE: 0.05 (training)

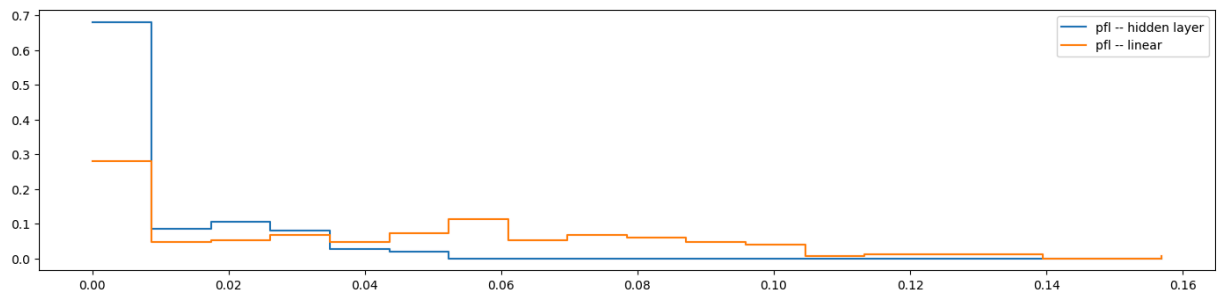
R2: 0.95, MAE: 0.04, RMSE: 0.05 (test)

More accurate, it is!

PFL Strikes Back

...And the improvement in terms of regret is remarkable

```
In [6]: r_ts_acc = util.compute_regret(prb, pfl_acc, data_ts.index.values, data_ts.v
util.plot_histogram(r_ts_acc, figsize=figsize, label='pfl -- hidden layer',
```

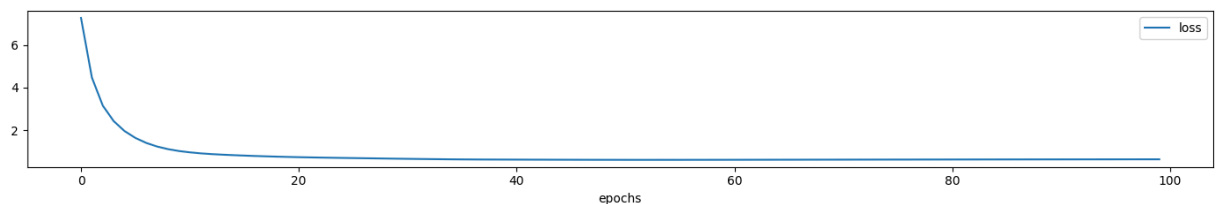


Mean: 0.009 (pfl -- hidden layer), 0.044 (pfl -- linear)

PFL Strikes Back

Let's what DFL could do with the same model coplexity

```
In [7]: spo_nonlinear = util.build_dfl_ml_model(input_size=1, output_size=nitems, pr
history = util.train_dfl_model(spo_nonlinear, data_tr.index.values, data_tr.v
util.plot_training_history(history, figsize=figsize_narrow, print_scores=False)
util.print_ml_metrics(spo_nonlinear, data_tr.index.values, data_tr.values, l
util.print_ml_metrics(spo_nonlinear, data_ts.index.values, data_ts.values, l
```



Training time: 49.9470 sec

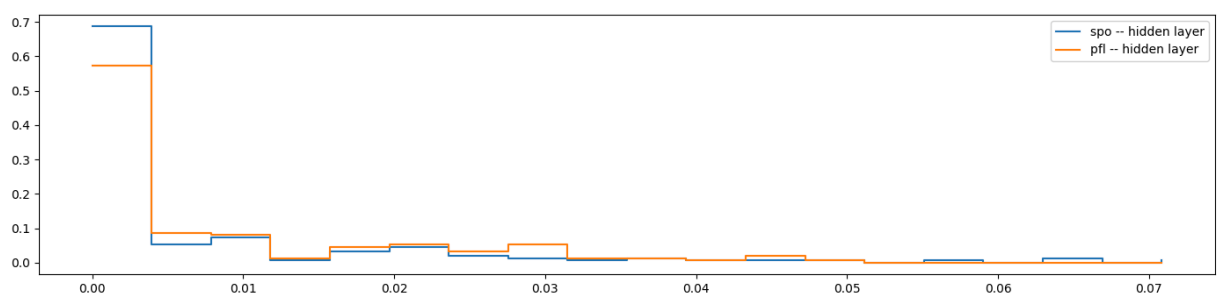
R2: -0.11, MAE: 0.2, RMSE: 0.24 (training)

R2: -0.08, MAE: 0.2, RMSE: 0.24 (test)

PFL Strikes Back

The gap between the approach is basically closed

```
In [8]: r_ts_spo_nonlinear = util.compute_regret(prb, spo_nonlinear, data_ts.index.v
fig = util.plot_histogram(r_ts_spo_nonlinear, figsize=figsize, label='spo --
fig.savefig('pfl_dfl_shallow.pdf')
```



Mean: 0.007 (spo -- hidden layer), 0.009 (pfl -- hidden layer)

Evening the Field

Can't we do anything about it?

- DFL predictions will always be off (more or less)
- ...But there are ways to make the approach faster

For example:

- You can use a relaxation, e.g. the [LP relaxation of a MILP](#)
- You can limit recomputation by [caching past solutions](#)
- You can warm start the DFL approach with the PFL weights

Let's see the last two tricks in deeper detail

Warm Starting and Solution Caching

Warm starting simple consists in using the PFL weights to *initialize* θ

Since accuracy is correlated with regret, this might accelerate convergence

Solution caching is applicable if *the feasible space is fixed*

I.e. to problems in the form:

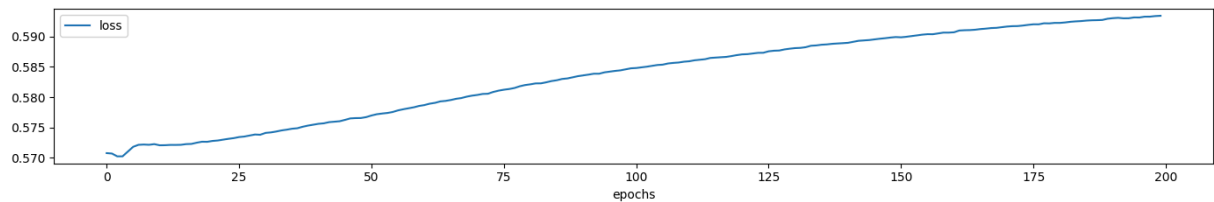
$$z^*(y) = \operatorname{argmin}_z \{f(z) \mid z \in F\}$$

- During training, we maintain a solution cache S
- Initially, we populate S with the true optimal solutions $z^*(y_i)$ for all examples
- Before computing $z^*(\hat{y})$ for the current prediction we flip a coin
- With probability p , we run the computation (and store any new solution in S)
- With probability $1 - p$, we solve instead $\hat{z}^*(\hat{y}) = \operatorname{argmin}_z \{f(z) \mid z \in S\}$

Speeding Up DFL

Let's use DFL *with linear regression*, a warm start, and a solution cache

```
In [9]: spo = util.build_dfl_ml_model(input_size=1, output_size=nitems, problem=prb,
history = util.train_dfl_model(spo, data_tr.index.values, data_tr.values, ep
util.plot_training_history(history, figsize=figsize_narrow, print_scores=False)
util.print_ml_metrics(spo, data_tr.index.values, data_tr.values, label='train')
util.print_ml_metrics(spo, data_ts.index.values, data_ts.values, label='test')
```



Training time: 21.6858 sec

R2: 0.54, MAE: 0.12, RMSE: 0.16 (training)

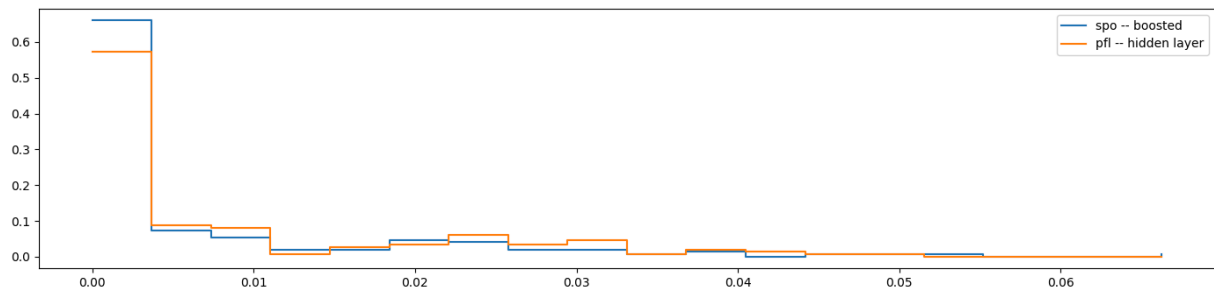
R2: 0.53, MAE: 0.12, RMSE: 0.17 (test)

The training time is still large, but much lower than our earlier DFL attempt

Speeding Up DFL

And the regret is even better than before!

```
In [10]: r_ts_spo = util.compute_regret(prb, spo, data_ts.index.values, data_ts.value
util.plot_histogram(r_ts_spo, figsize=figsize, label='spo -- boosted', data2
```



Mean: 0.007 (spo -- boosted), 0.009 (pfl -- hidden layer)

- We are matching the more complex PFL model with a simple linear regressor
- ...And the training time is much better than before

Reflecting on What we Have

Therefore, DFL gives us at least *two benefits*

First, it can lead to *lower regret* compared to a prediction-focused approach

- As the models become more complex we have diminishing returns
- ...But for some applications every little bit counts

Second, it may allow using *simpler ML models*

- Simple models are faster to evaluate
- ...But more importantly they are *easier to explain*
- E.g. we can easily perform feature importance analysis

Intuitively, DFL works best *where PFL has estimation issues*

Can we exploit this fact to maximize our advantage?